Full Custom Layout of Neural Network Processing Element Using Push Pull D Flip Flop and Modified Carry Look Ahead Adder

Adiwena Putra
University Center of Excellence on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

Trio Adiono
University Center of Excellence on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

2021 International Symposium on Electronics and Smart Devices (ISESD) | 978-1-6654-4146-9/21/$31.00 ©2021 IEEE | DOI: 10.1109/ISESD53023.2021.9501832

***Abstract*—This paper presents a full custom design of a 4-bit ANN processing element, implemented using 120 nm technology. The processing element consists of register, adder, and multiplier. The register is implemented using push-pull D flip-flop architecture, the adder is implemented using modified carry look-ahead architecture, while the multiplier is implemented using the parallel architecture of the radix-4 booth. The processing element is fully functional and has been simulated with a 1 GHz clock frequency. The total area of the processing element is 3752.6 μm^2 with the worst path delay of 305 ps.**

***Index Terms*—booth multiplier, push pull D flip-flop, carry look ahead adder, artificial neural network**

I. Introduction

ANN is a computational model of the biological brain. Its implementation plays a key role in the development of AI technology. ANN consists of a set of layers, where at each layer there is a perceptron which is the smallest processing element in ANN. Perceptron is basically the MAC operation (multiply-accumulate) and pass the result to the activation function. In the recent development of AI, ANN can require thousands to millions of perceptron. This means that the implementation of ANN is computationally expensive and consume high amount of power.

To address this problem, current research focuses to design the hardware accelerator for ANN computation. In many cases, the hardware accelerator is implemented using FPGA or ASIC. While FPGA is one of the interesting platform because of its flexibility to configure the logic fabric, FPGA hardware accelerator still suffering from the resource optimization problem and low clock frequency. In contrary, ASIC implementation, despite its lack of flexibility, can offer optimize resource and higher clock frequency. Thus, ASIC implementation can give higher performance compare to FPGA.

Moreover, there are 2 popular design methodology in ASIC: 1) standard cell technology and 2) full custom technology. In standard cell approach, we use the standard cell library, where we have the pre-designed building blocks to build the entire circuit. While in full custom approach, we designed the chip layout from scratch without using any pre-designed building blocks. However, ASIC for AI accelerator just getting popular in recent years. The problem is the standard cell library that we had is not really suitable to build the high performance AI accelerator. In this work, we design the full custom layout for one of the most basic components in every AI accelerator.

One of the key element in hardware accelerator design is the dataflow [8]. The dataflow dictates the flow of data and orchestrates the memory management in accelerator. There are several choice of dataflow which can optimize the reuse of selected data type (weight, input activations, output activations). Out of all dataflow, row stationary is the dataflow that can optimize the overall data reuse and provides the lowest energy consumption [1], [2].

To implement row stationary dataflow, the computation is done in hundred of processing elements. Each processing elements have its own ALU (MAC) and register file (cache memory) to store activations, weight, and partial sum. In this paper, we propose full custom layout implementation of row stationary processing element for ASIC implementation. There are three major blocks in processing element: register, multiplier, and adder. We will discuss each architecture of the major blocks in processing element as well as the layout implementation. The main contribution of this paper is as following.

- We propose the full custom layout of processing element. The novelty in this full custom layout is located in the choice of each building blocks. We choose the combination of state of the art components to build high performance layout processing elements with smallest latency.
- We deconstruct major block in processing element and design it separately using state of the art component.

The rest of the paper is structured as follows. Section II provides the architecture and layout implementation of each major building block in processing element. Section III describe the final result and the layout performance in simulation. Finally, section IV conclude the result of this works and the potential improvement for future works.

978-1-6654-4146-9/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:37:14 UTC from IEEE Xplore. Restrictions apply.

II. Design and Implementation

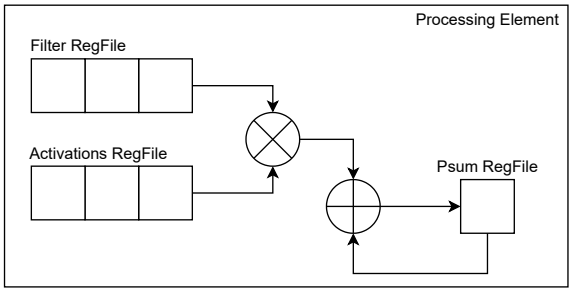


Fig. 1. Schematic of Push Pull D Flip-Flop

Figure 1 shows the basic processing element which can compute dot product operation between 2 vector. We deconstruct our layout into 3 major building block. First, the register file, we use state of the art push pull D flip flop to implement the register for processing element. Second, the adder, we use the carry look ahead adder for the fastest computation, we also modify the carry look ahead architecture in order to minimize the area of layout. Third, the multiplier, we use parallel architecture of booth radix-4 multiplier.

A. D Flip-Flop and Register

D flip-flop is the basic unit that will implement all kinds of register in processing element. There are several architectures to implement the D flip-flop with its own advantages. The metric that we used to determined the architecture will be the speed and the robustness of D flip-flop. According to the comparative study of D flip-flop architectures [6] the push-pull isolation D flip-flop will have the advantage in term of speed and robustness. We do a little modification in order to add reset functionality in D flip-flop. Figure 2 is the schematic of push-pull isolation D flip-flop that which we implement using 120 nm technology.

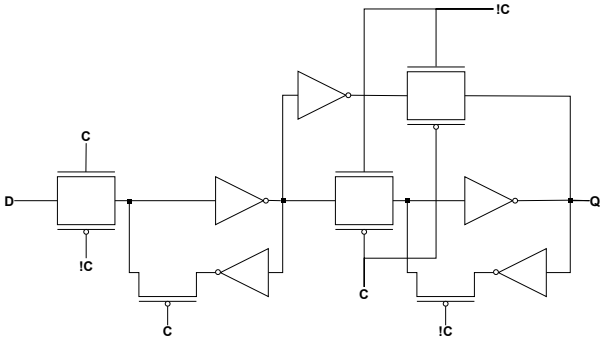


Fig. 2. Schematic of Push Pull D Flip-Flop

For the rest of this paper, we use the transistor size describe in Table I. The size will be adjusted accordingly for another gate in order to achieve the balance between rising and falling time.

Figure 3 is the layout implementation for one D flip-flop using the push-pull isolation architecture. The total area and performance of D flip-flop according to the simulations are shown in Table II.

TABLE I
CMOS Dimension

Mos Transistor	W	L
NMOS	0.60 μm	0.12 μm
PMOS	1.50 μm	0.12 μm

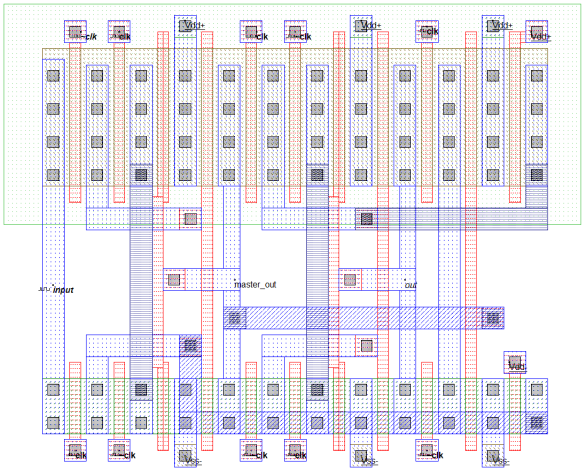


Fig. 3. Layout of Push Pull D Flip-Flop

TABLE II
Total Area and Performance of D Flip Flop

Metric	Result
Total Area	23.1 μm^2
T_{cq}	6 ps
T_{su}	12 ps
T_h	$\leq$ 1 ps

Using this basic layout of D flip-flop, we implement the register and shift register for processing element. Figure 4 and Figure 5 show the layout implementation of 4 bit register and 4 bit shift register. The total area of 4 bit register is 121.67 μm^2 and the total area of shift register is 430.92 μm^2.

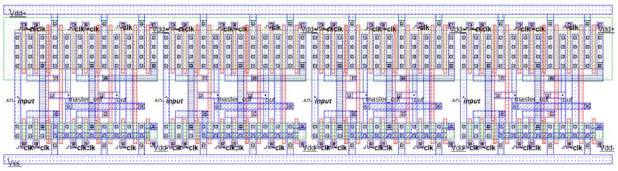


Fig. 4. Layout of 4 Bit Register

B. Adder

The adder is implemented using the modified carry look ahead architecture. The modification is done in order to minimize the size of overall adder [7]. The problem of adder can be define as $S = A + B$, where A and B are the first and

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:37:14 UTC from IEEE Xplore. Restrictions apply.

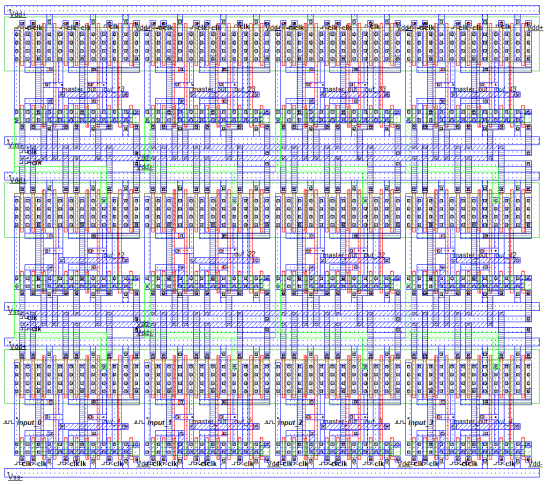


Fig. 5. Layout of 4 Bit Shift Register

second operand, and S is the final sum result. For carry look ahead adder, we need to get the propagate bit and generate bit in order to generate the carry bit. The propagate bit and generate bit can be obtained using following equations.

$$\begin{aligned} P_i &= A_i \oplus B_i \\ G_i &= A_i B_i \end{aligned} \tag{1}$$

where:

P_i = i-th bit of propagate bit
G_i = i-th bit of generate bit
A_i = i-th bit of first operand
B_i = i-th bit of second operand

In typical carry look ahead adder, the carry can be obtained using following equations.

$$\begin{aligned} C_1 &= G_0 + P_0 C_{in} \\ C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_{in} \\ C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{in} \\ C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{in} \end{aligned} \tag{2}$$

In the modified carry look ahead adder, the carry will be obtain using maximum 2 stage of NAND gate. Following is the modified formula to obtain carry in modified carry look ahead adder.

$$\begin{aligned} C_0 &= input\ carry \\ C_1 &= \overline{\overline{G_0}\ \overline{P_0 C_{in}}} \\ C_2 &= \overline{\overline{G_1}\ \overline{P_1 G_0}\ \overline{P_1 P_0 C_{in}}} \\ C_3 &= \overline{\overline{G_2}\ \overline{P_2 G_1}\ \overline{P_2 P_1 G_0}\ \overline{P_2 P_1 P_0 C_{in}}} \\ C_4 &= \overline{\overline{G_3}\ \overline{P_3 G_2}\ \overline{P_3 P_2 G_1}\ \overline{P_3 P_2 P_1 G_0}\ \overline{P_3 P_2 P_1 P_0 C_{in}}} \end{aligned} \tag{3}$$

Figure 6 shows the architecture of 4-bit modified carry look ahead adder. Here, MPFA (Modified Partial Full Adder) can be implemented using the circuit in Figure 7, while C1, C2, C3 and C4 compute blocks are the circuit which implement

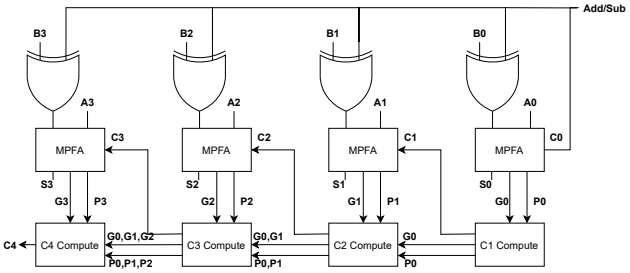


Fig. 6. Architecture of Modified Carry Look Ahead Adder

the Equation 3. Using this modified architecture of carry look ahead, the worst path delay of adder will only contain maximum of 3 gate delays. Figure 8 and Figure 9 shows the layout of MPFA and layout of overall carry look ahead adder. The total area of modified CLA is 473.68 μm^2 and the worst delay of this circuit is 45 ps.

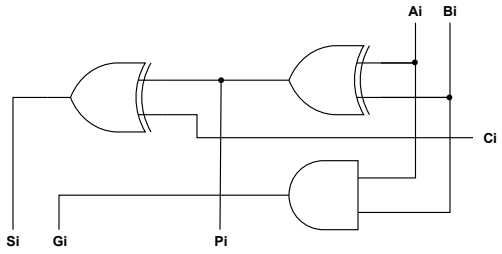


Fig. 7. Modified Partial Full Adder Circuit

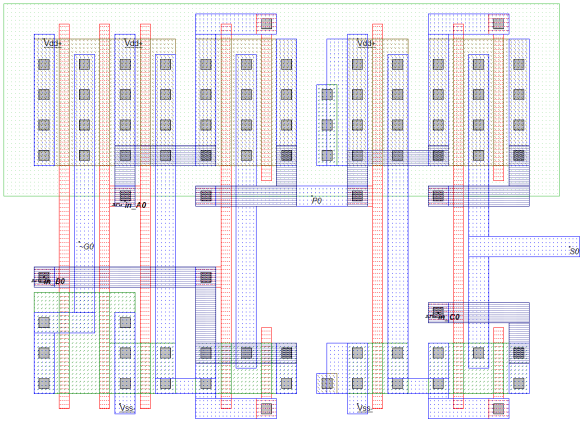


Fig. 8. Layout of Modified Partial Full Adder

C. Multiplier

The multiplier is implemented using booth radix-4 algorithm, in addition we also use parallel architecture so the result of multiplication can be obtained only in one clock cycle [9]. Figure 10 shows the overall architecture for parallel booth radix-4 multiplier. The architecture consist of booth recoder block, mux, and adder. Booth recoder block is the circuit which activate a sets of signal (sel, zero, add/sub) to control

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:37:14 UTC from IEEE Xplore. Restrictions apply.

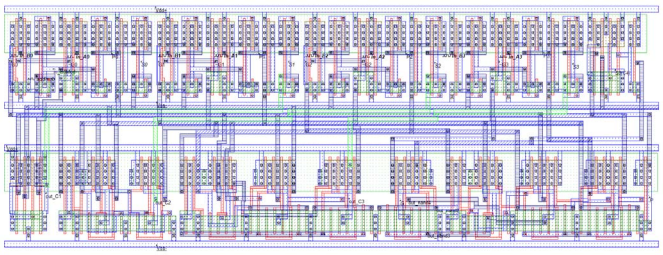


Fig. 9. Layout of Modified Carry Look Ahead Adder

the flow of data through the mux, while the adder we use for multiplier circuit is the same modified carry look ahead adder.

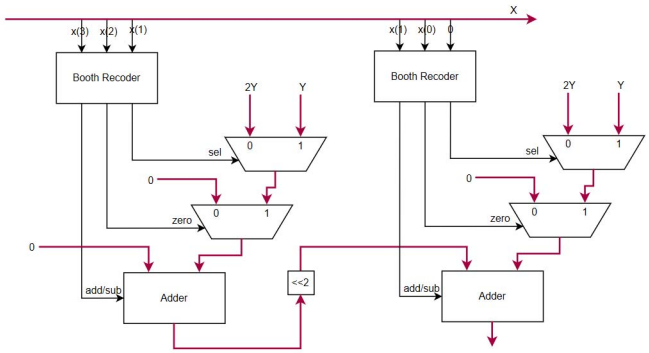


Fig. 10. Layout of Modified Carry Look Ahead Adder

In the layout, the implementation of two blocks booth recoder in Figure 10 is combined together, the same thing is also apply in four blocks of 4-bit mux. Figure 11 shows the layout of two blocks booth recoder combine into one layout, Figure 12 shows the layout of one 4-bit mux, and Figure 13 shows the layout of overall booth multiplier (the combination of 2 modified carry look ahead adder, 2 recoder block, and 4 block of mux). The total area of overall booth multiplier is 1665.07 μm^2, while the worst delay is 201 ps.

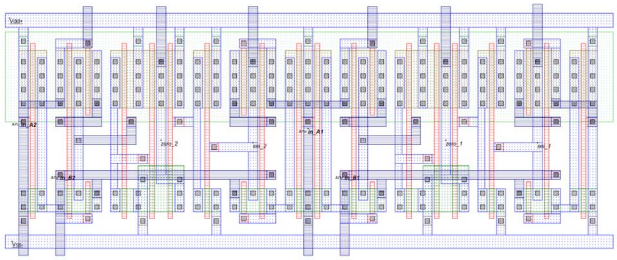


Fig. 11. Layout of Booth Recoder

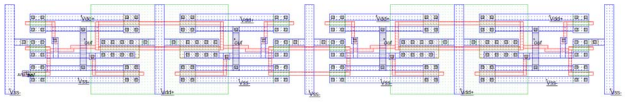


Fig. 12. Layout of 4 Bit Mux

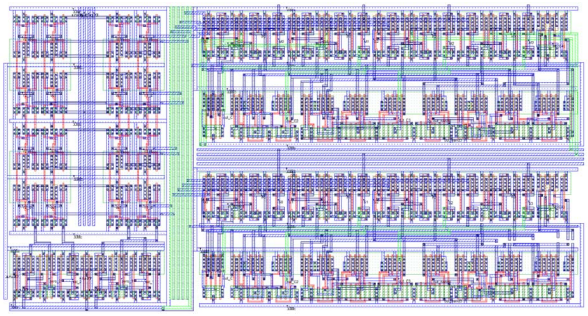


Fig. 13. Layout of Booth Multiplier Radix 4

III. Result

A. Floor Planning

Each of the building block that has been described above, will be joined together in order to get fully functional processing element that implement the MAC operation. Figure 14 and Figure 15 show the floor planning and the layout implementation of overall processing element.

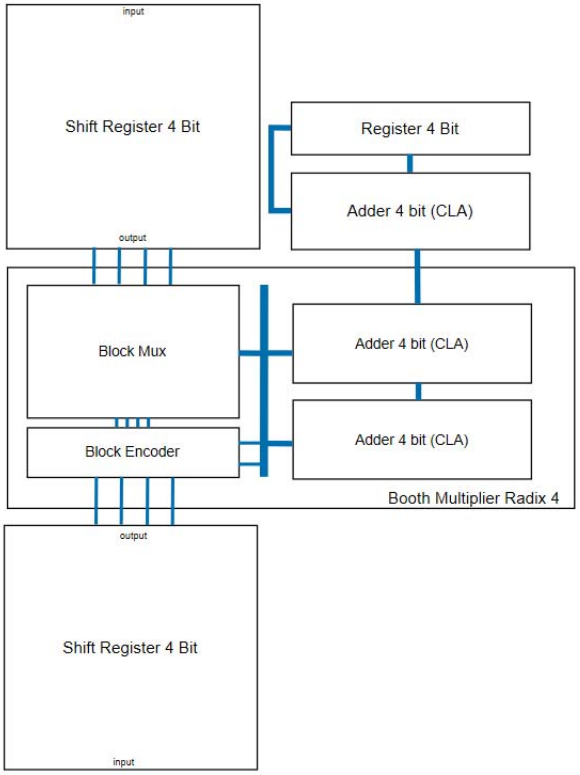


Fig. 14. Floor Planning of Processing Element

The layout has been simulated using several test cases. Figure 16 is the snippet of simulation result using microwind 2 software. Functionality, the processing element is working properly. In the test case in Figure 16 the input is given on times stamp = 2 ns. The X_4, X_3, X_2, X_1 bits and Y_4, Y_3, Y_2, Y_1 bits are the 4-bit outputs of shift register that

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:37:14 UTC from IEEE Xplore. Restrictions apply.

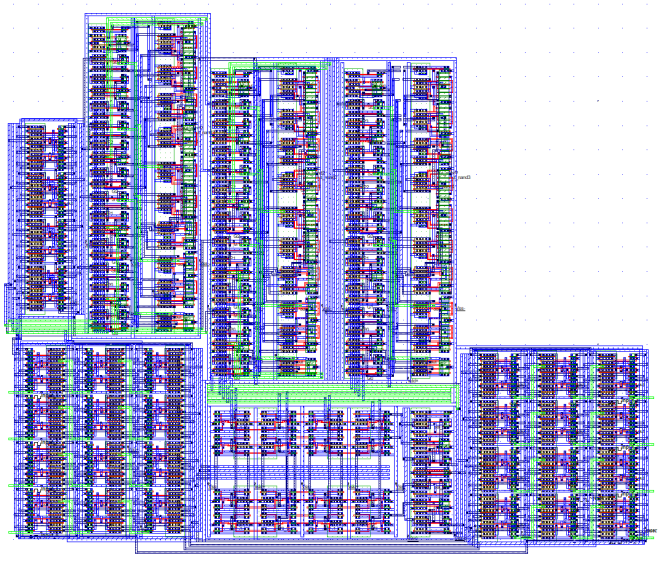


Fig. 15. Layout of Processing Element

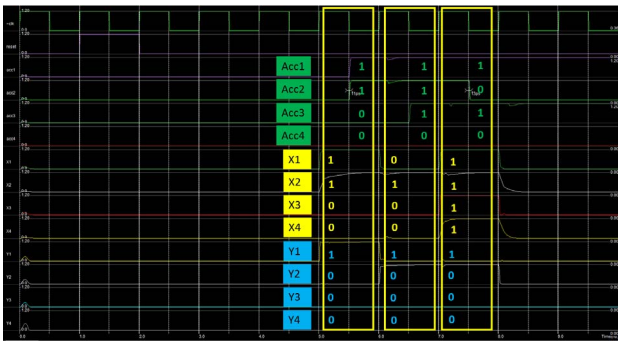


Fig. 16. Simulation Results of Processing Element Computation

directly will go to the multiplier. For the test case shown in Figure 16, we compute the dot product between these 2 vectors.

$$\begin{bmatrix} 3 & 2 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \end{bmatrix} \tag{4}$$

The final result is shown in bit $Acc_1, Acc_2, Acc_3, Acc_4$ bits.

From the simulation, the total area of 1 processing element is 3752.6 μm^2 and the worst path delay in entire circuit is 305 ps. The total latency for computing the dot product between 2 vectors (each with 3 elements) is 3.5 clock cycle. Latency is calculated from the first-time input is given to the shift register until the first-time output appears in 4-bit register. The worst path delay is path from the start of multiplier until the end of adder that will update the 4-bit register.

B. Comparison With Similar Works

In this section, we will compare our results with similar result from other works. To the best of author knowledge, there is no other paper which discusses the layout implementation of overall processing element. Thus, we will make comparison of our works part by part for multiplier and D flip flop implementation.

TABLE III
MULTIPLIER COMPARISON

Metric	**[4]**	**[5]**	**this work**
Multiplier Method	Radix 4 Booth Multiplier	Radix 4 Booth Multiplier	Radix 4 Booth Multiplier
Adder	Ripple Carry Adder	Carry Look Ahead Adder	Modified Carry Look Ahead Adder
Bit	8	8	4
Area (μm^2)	41501	76360	1665
Technology (nm)	130	130	120
Maximum Frequency	2.50 MHz	3.70 MHz	4.98 GHz

TABLE IV
D FLIP FLOP COMPARISON

Metric	**[12]**	**[13]**	**this work**
Method	TSPC	MTSPC	Push Pull
Transistor Count	11	13	18
Delay	98 ps	64 ps	12 ps

Table III shows the comparison of our multiplier to other works. For similar comparison, we search multiplier which is implemented using Radix 4 Booth Algorithm. The different between multiplier design is located in the choice of adder. In paper [4], the author uses ripple carry adder for multiplier. The ripple carry adder has advantage of simple design and relatively small area, but it suffers from high propagation delay which made the maximum clock frequency to be low. In paper [5], the author uses standard carry look ahead adder which is faster adder compare to [4], bit with the trade off large design area. In our work, we use modified carry look ahead adder which optimize the area and speed of our multiplier. In addition, we also use push pull D flip flop which is more robust and fast compare to standard D flip flop used in [4] and [5]. With the combination of push pull D flip flop and modified architecture of carry look ahead, we manage to achieve maximum clock frequency up to 4.98 GHz. Still, our multiplier is implemented in 4 bit while [4] and [5] use 8 bit multiplier. In future work, the number of bit in multiplier will be upgraded for more fair comparison.

Table IV shows the comparison of our D flip flop compare to other D flip flop implementation. In this table, delay is defined as setup time in D flip flop. In paper [12], the author use true single phase clocking (TSPC) D flip flop to achieve 98 ps delay which is already faster compare to standard implementation. This result then is further developed in paper [13] which implement the modified true single phase clocking (MTSPC) D flip flop and managed to achieve 64 ps delay. In our implementation, we use push pull D flip - flop which can

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:37:14 UTC from IEEE Xplore. Restrictions apply.

achieve significantly smaller delay up to 12 ps with the trade off more transistor count to implement the D flip flop.

IV. Conclusion

The 4-bit processing element is designed using 120 nm CMOS technology and the results have been tested. We implement the push-pull D flip-flop for the register and the modified version of carry look ahead for the adder. We use the radix-4 booth algorithm for multiplier and modified it with parallel architecture in order to increase the throughput and speed of the multiplier. The final layout of processing element has the total area of 3752.6 μm^2 with worst path delay 305 ps.

References

[1] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," IEEE Journal of Solid-State Circuits, vol. 52, no. 1, pp. 127–138, 2017.

[2] Y.-H. Chen, T.-J. Yang, J. S. Emer, and V. Sze, "Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices," IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 9, no. 2, pp. 292–308, 2019

[3] C. M. Maxfield, "FPGA vs. ASIC Designs," FPGAs: Instant Access, pp. 61–73, 2008.

[4] Y. F. Arthanto, S. I. Ibad, and T. Adiono, "High Speed Full Custom Parallel Multiplier Based on Radix-4 Booth," 2019 International Symposium on Electronics and Smart Devices (ISESD), 2019.

[5] T. Tafriyana, L. K. Sari, and T. Adiono, "Fully Combinational 8 × 8 Bits Multiplier Using 130 nm Technology," 2019 International Conference on Electrical Engineering and Informatics (ICEEI), 2019.

[6] H. Kumar, A. Kumar, and A. Islam, "Comparative analysis of D flip-flops in terms of delay and its variability," 2015 4th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO) (Trends and Future Directions), 2015.

[7] Yu-Ting Pai and Yu-Kumg Chen, "The fastest carry lookahead adder," Proceedings. DELTA 2004. Second IEEE International Workshop on Electronic Design, Test and Applications, 2004, pp. 434-436, doi: 10.1109/DELTA.2004.10071.

[8] Y.-H. Chen, J. Emer, and V. Sze, "Using Dataflow to Optimize Energy Efficiency of Deep Neural Network Accelerators," IEEE Micro, vol. 37, no. 3, pp. 12–21, 2017.

[9] T. Adiono, Perancangan Rangkaian Terintegrasi Digital. Bandung, Jawa Barat: ITB Press, 2017.

[10] D. Kalaiyarasi and M. Saraswathi, "Design of an Efficient High Speed Radix-4 Booth Multiplier for both Signed and Unsigned Numbers," 2018 Fourth International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics (AEEICB), 2018, pp. 1-6, doi: 10.1109/AEEICB.2018.8480959.

[11] E. K. Surendran and P. R. Antony, "Implementation of fast multiplier using modified Radix-4 booth algorithm with redundant binary adder for low energy applications," 2014 First International Conference on Computational Systems and Communications (ICCSC), 2014.

[12] H. Ashwini, S. Rohith, and K. A. Sunitha, "Implementation of high speed and low power 5T-TSPC D flip-flop and its application," 2016 International Conference on Communication and Signal Processing (ICCSP), 2016.

[13] J. Shaikh and H. Rahaman, "High speed and low power preset-able modified TSPC D flip-flop design and performance comparison with TSPC D flip-flop," 2018 International Symposium on Devices, Circuits and Systems (ISDCS), 2018.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 13:37:14 UTC from IEEE Xplore. Restrictions apply.